

# RAPPORT DE PROJET

Plateforme de gestion de demandes de développement Web et Mobile  
avec profils freelances

*Réalisé dans le cadre du Master 1 — Génie Logiciel*

Technologies : HTML5, CSS3, JavaScript, Express.js, MySQL

**Sommaire**

# 1. Introduction

Le développement web et mobile est aujourd'hui un secteur en forte demande, mais la mise en relation entre clients porteurs de projets et développeurs qualifiés reste souvent difficile, en particulier pour les jeunes développeurs qui cherchent à se constituer une expérience professionnelle vérifiable.

Ce projet propose la conception et la réalisation d'une plateforme web permettant à des clients de soumettre des demandes de développement (site web ou application mobile), et à des freelances de constituer un profil professionnel complet (CV, compétences, portfolio, liens vers GitHub, LinkedIn et Upwork) afin de répondre à ces demandes.

Ce document présente le contexte du projet, l'analyse des besoins, la conception de la base de données, l'architecture technique retenue, les mesures de sécurité mises en œuvre, la stratégie de déploiement, ainsi que les perspectives d'évolution vers l'intelligence artificielle et une application mobile.

## 2. Contexte et problématique

Dans le cadre de ma formation en Master 1 Génie Logiciel, et dans une démarche de professionnalisation en tant que développeur full stack freelance, j'ai souhaité concevoir un outil concret qui réponde à un besoin réel : centraliser en un seul endroit la gestion des demandes clients, mon profil professionnel et mes réalisations.

### 2.1 Problématique

Comment concevoir une plateforme sécurisée, capable de gérer efficacement des demandes de projets web/mobile, tout en valorisant le profil et les compétences du développeur freelance qui les traite ?

### 2.2 Objectifs du projet

- Permettre à un client de soumettre facilement une demande de site web ou d'application mobile.
- Offrir au freelance un profil professionnel complet (bio, compétences, portfolio, CV, liens GitHub/LinkedIn/Upwork).
- Assurer la sécurité des données (authentification, hachage des mots de passe, protection contre les injections SQL).
- Déployer la plateforme sur un serveur accessible publiquement, avec HTTPS.
- Préparer l'intégration d'un module d'intelligence artificielle pour le matching automatique demandes/compétences.
- Prévoir une extension vers une application mobile.

## 3. Technologies utilisées

| Couche          | Technologie                 | Rôle                                                 |
|-----------------|-----------------------------|------------------------------------------------------|
| Frontend        | HTML5 / CSS3 / JavaScript   | Interface utilisateur, formulaires, interactivité    |
| Backend         | Node.js + Express.js        | API REST, logique métier, routage                    |
| Base de données | MySQL 8                     | Stockage relationnel des données                     |
| Sécurité        | bcrypt, JWT, Helmet, dotenv | Authentification, chiffrement, protection HTTP       |
| Serveur         | Nginx + PM2 (VPS Linux)     | Reverse proxy, HTTPS, disponibilité de l'application |

### 3.1 Outils de développement utilisés

Au-delà des technologies du projet, plusieurs outils ont accompagné le développement, les tests et le déploiement de la plateforme :

| Outil                   | Utilisation                                                                                       |
|-------------------------|---------------------------------------------------------------------------------------------------|
| Visual Studio Code      | Éditeur de code principal (extensions ESLint, Prettier, MySQL, Docker)                            |
| Docker / Docker Compose | Conteneurisation de l'application Node.js et de la base MySQL pour un environnement reproductible |
| Git & GitHub            | Gestion de versions, historique des commits, hébergement du code source                           |
| Postman                 | Test et documentation des routes de l'API REST Express                                            |
| MySQL Workbench         | Conception visuelle du schéma et administration de la base de données                             |
| npm                     | Gestion des dépendances et des scripts du projet Node.js                                          |
| Nginx                   | Reverse proxy et terminaison HTTPS en production                                                  |
| PM2                     | Supervision et redémarrage automatique du processus Node.js en production                         |

L'usage de Docker permet notamment d'assurer que l'environnement de développement (backend Node.js + base MySQL) soit identique entre la machine locale et le serveur de production, ce qui réduit les erreurs liées aux différences de configuration.

## 4. Analyse des besoins

### 4.1 Acteurs du système

| Acteur         | Rôle principal                                                                                                    |
|----------------|-------------------------------------------------------------------------------------------------------------------|
| Client         | Crée un compte, publie une demande de projet, échange avec les freelances, valide une proposition, laisse un avis |
| Freelance      | Crée un profil professionnel, consulte les demandes, envoie des propositions, échange avec les clients            |
| Administrateur | Supervise les comptes, modère les contenus, consulte les statistiques                                             |

### 4.2 Principales fonctionnalités

- Inscription / connexion sécurisée (client, freelance, admin).
- Gestion du profil freelance : bio, compétences, tarif, CV, portfolio, liens externes.
- Publication et suivi d'une demande de projet.
- Système de propositions (offres de prix et de délai).
- Messagerie entre client et freelance autour d'une demande.
- Système d'avis et de notation après la réalisation d'un projet.
- Suivi des paiements liés à un projet.

## 5. Conception de la base de données

Le modèle de données repose sur 10 entités principales, reliées entre elles par des clés étrangères garantissant l'intégrité référentielle. Le script SQL complet est fourni en fichier séparé (schema\_base\_donnees.sql).

| Table                            | Description                                                                  |
|----------------------------------|------------------------------------------------------------------------------|
| utilisateurs                     | Comptes clients, freelances et administrateurs                               |
| profils_freelance                | Informations professionnelles : bio, CV, tarif, liens GitHub/LinkedIn/Upwork |
| competences / profil_competences | Référentiel de compétences et liaison N-N avec les profils                   |
| portfolio                        | Projets déjà réalisés par un freelance                                       |

|                |                                                                  |
|----------------|------------------------------------------------------------------|
| demandes       | Projets postés par les clients (site web, application mobile...) |
| propositions   | Offres soumises par les freelances sur une demande               |
| messages       | Échanges entre client et freelance                               |
| avis           | Notation et commentaire après un projet terminé                  |
| paiements      | Suivi des paiements liés à une demande                           |
| suggestions_ia | Résultats du futur module de matching intelligent                |

Les relations clés sont : un utilisateur freelance possède un seul profil (1-1), un profil peut avoir plusieurs compétences et plusieurs éléments de portfolio (1-N), une demande peut recevoir plusieurs propositions (1-N), et chaque demande terminée peut donner lieu à un avis et un paiement.

## 6. Sécurité mise en place

- Hachage des mots de passe avec bcrypt (jamais de mot de passe stocké en clair).
- Authentification par jetons JWT avec expiration.
- Requêtes SQL préparées (mysql2) pour prévenir les injections SQL.
- Validation systématique des entrées utilisateur côté serveur (express-validator).
- En-têtes HTTP sécurisés via Helmet.js.
- Limitation du nombre de requêtes (express-rate-limit) contre les attaques par force brute.
- Variables sensibles (mots de passe BDD, clés secrètes) stockées dans un fichier .env, exclu du dépôt Git.
- Chiffrement du trafic via HTTPS (certificat Let's Encrypt).

## 7. Déploiement

L'application est déployée sur un serveur privé virtuel (VPS) sous Linux. Node.js exécute le backend Express, supervisé par PM2 pour assurer un redémarrage automatique en cas d'incident. Nginx agit comme reverse proxy devant l'application et gère la terminaison HTTPS via un certificat Let's Encrypt. Des sauvegardes automatiques de la base MySQL sont programmées quotidiennement, et un service de monitoring externe surveille la disponibilité du site.

## 8. Intégration de l'intelligence artificielle (proposition)

Afin d'améliorer l'expérience utilisateur, un module d'intelligence artificielle est proposé pour analyser automatiquement le texte d'une demande de projet et en extraire les compétences techniques requises, puis recommander les freelances les plus pertinents en fonction de leur profil et de leurs avis précédents.

### 8.1 Fonctionnement envisagé

- Le client rédige la description de son projet en langage naturel.
- Un modèle de traitement du langage (NLP) extrait les technologies et le type de projet évoqués.
- Le système compare ces éléments aux compétences enregistrées dans les profils freelances.
- Une liste de freelances recommandés est proposée, avec une estimation indicative de budget et de délai.

Les résultats de cette analyse seraient stockés dans la table suggestions\_ia déjà prévue dans le schéma de base de données, ce qui permet d'ajouter ce module sans modifier la structure existante.

## 9. Perspective : application mobile

Une extension de la plateforme vers le mobile est envisagée afin de permettre aux clients et aux freelances de suivre leurs demandes, propositions et messages depuis un smartphone. L'application mobile s'appuierait sur la même API REST Express déjà développée pour le site web, ce qui évite toute duplication de la logique métier.

| Critère          | Choix recommandé                                                    |
|------------------|---------------------------------------------------------------------|
| Framework        | React Native ou Flutter (développement multiplateforme iOS/Android) |
| Communication    | Réutilisation de l'API REST Express existante                       |
| Authentification | Jetons JWT partagés avec la version web                             |
| Notifications    | Firebase Cloud Messaging pour les nouvelles propositions/messages   |

## 10. Conclusion

Ce projet a permis de concevoir une plateforme complète de mise en relation entre clients et développeurs freelances, en appliquant des principes solides de sécurité, une architecture claire (HTML/CSS/JS, Express.js, MySQL) et une base de données structurée et évolutive.

Les perspectives d'évolution — intégration d'un module d'intelligence artificielle pour le matching automatique et développement d'une application mobile — s'appuient directement sur l'architecture déjà mise en place, ce qui garantit une continuité technique cohérente pour les prochaines étapes du projet.